

TaskMaster API & Dashboard

Role: Software Developer

Difficulty: junior

Project Type: fullstack

Duration: 14 days

Focus: api_development

Candidate: Rahul Sharma

Project Overview

Build a task management API with a React/Next.js dashboard, using JWT auth and role based access.

Technologies

- JavaScript
- TypeScript
- React
- Next.js
- Node.js
- Express.js
- Laravel (PHP)
- Tailwind CSS
- MongoDB
- JWT

Requirements

- Use existing skills, add new learning, measurable outcome

Objectives

- Implement secure CRUD API
- Create role based dashboard
- Document API
- Write tests
- Deploy locally

Functional Requirements

- User register/login
- JWT auth
- Create/read/update/delete tasks
- Admin can delete any task
- Frontend displays tasks
- Export tasks to PDF

Non-Functional Requirements

- RESTful design
- Input validation
- Error handling
- JWT expiration
- Responsive UI
- Unit tests

Modules

Auth

User registration & login

- JWT issuance
- Password hashing

Task

Task CRUD

- Create, read, update, delete
- Owner & admin access

User

User profile

- Role assignment
- Profile view

API

Express routes

- Middleware, validation, error handling

Frontend

Next.js dashboard

- Task list, form, PDF export

Deployment

Local Docker

- Dockerfile, docker compose

Implementation Plan

Setup

- Initialize repo, create backend & frontend folders
- Set up Node, Express, Next, Tailwind

Auth

- Create register & login routes
- Implement JWT, bcrypt

Task CRUD

- Define Task model, routes, middleware
- Add role checks

Frontend

- Build task list page, form, PDF export

Testing

- Write unit tests for API, integration tests for auth

Deployment

- Create Dockerfile, docker compose, run locally

Evaluation Criteria

API Design (25%)

RESTful, clear endpoints

Security (20%)

JWT, role checks, hashing

Code Quality (20%)

Clean, modular, comments

Testing (15%)

Coverage >80%

Documentation (10%)

API docs, README

UI/UX (10%)

Responsive, usable

Deliverables

- Source code
- README
- API docs
- Test report

Suggested Folder Structure

- backend
- frontend
- docs
- tests

API Endpoints

POST /api/auth/register

Create user

POST /api/auth/login

Login & JWT

GET /api/tasks

List tasks

POST /api/tasks

Create task

GET /api/tasks/:id

Get task

PUT /api/tasks/:id

Update task

DELETE /api/tasks/:id

Delete task

GET /api/tasks/export

Export PDF

Database Design

users

Store user data

- _id
- email
- passwordHash
- role

tasks

Store tasks

- _id
- title
- description
- ownerId
- status
- createdAt

Testing Plan

- Unit tests for auth & task routes
- Integration tests for JWT flow
- Frontend component tests

Candidate Fit

Rahul's JS/TS, React, Next.js, Express, JWT, and MongoDB skills align with the project, while the task introduces role based auth and PDF export for growth.